HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA AN TOÀN THÔNG TIN



BÁO CÁO HỌC PHẦN

Đánh giá an toàn và bảo vệ dữ liệu người dùng ứng dụng OWASP Juice Shop v20.2.0

Học phần: DevOps và DevSecOps (SEC4349)

Thành viên	MSSV	Lỗi phụ trách
Nguyễn Toàn Thắng	B26CHAT039	F01 — SQL injection đăng nhập
Phạm Thị Minh Thúy	B26CHAT042	F02 — Poison Null Byte
Đỗ Đức Quốc Anh	B26CHAT001	F03 — lộ endpoint /metrics
Hồ Anh Vũ	B26CHAT048	F04 — cấu hình an toàn container

Năm học 2026

Mục lục

Danh mục từ viết tắt

1. Giới thiệu và phạm vi

2. Ứng dụng Juice Shop và dữ liệu cần bảo vệ

2.1. Nghiệp vụ đăng ký và đăng nhập

2.2. Nghiệp vụ tải tệp tài liệu

3. Môi trường triển khai và yêu cầu an toàn tối thiểu

3.1. Công cụ sử dụng

3.2. Chính sách và tiêu chuẩn đối chiếu

4. Quy trình DevSecOps và các cổng kiểm soát

4.1. Quét và phát hiện lỗi — kết quả thực tế

4.2. Bảng kết quả hợp nhất

4.3. Giám sát khi vận hành (pha Operate)

4.4. Lệnh quét (tái lập)

5. Bốn lỗi an toàn: phát hiện và xử lý

5.1. F01 — SQL injection tại đăng nhập

5.2. F02 — Poison Null Byte làm lộ tệp sao lưu

5.3. F03 — Lộ endpoint /metrics không xác thực

5.4. F04 — Sai cấu hình an toàn của container

6. Kết luận

7. Tài liệu tham khảo

Phụ lục A — Mã nguồn chính sách Dockerfile (docker_security.rego)

Phụ lục B — Mã nguồn chính sách docker-compose (compose.rego)

Phụ lục C — Giải thích tiêu chuẩn và điều khoản tham chiếu

Danh mục từ viết tắt

Bảng 1. Danh mục từ viết tắt

Viết tắt	Giải nghĩa
SQLi	SQL Injection — tiêm nhiễm câu lệnh SQL
JWT	JSON Web Token — token phiên
SAST	Phân tích tĩnh mã nguồn (Static Application Security Testing)
CI/CD	Tích hợp / triển khai liên tục
OPA	Open Policy Agent (ngôn ngữ Rego, công cụ Conftest)
GDPR	Quy định bảo vệ dữ liệu của EU (2016/679)
PCI-DSS	Chuẩn an toàn dữ liệu thẻ thanh toán
CIS	Center for Internet Security (CIS Docker Benchmark)
CWE / OWASP	Phân loại điểm yếu phần mềm / tổ chức an toàn ứng dụng web
UID	Mã định danh người dùng hệ điều hành

1. Giới thiệu và phạm vi

Báo cáo đánh giá an toàn ứng dụng **OWASP Juice Shop v20.2.0** dưới góc độ bảo vệ dữ liệu người dùng qua **bốn tình huống**. Bốn thành viên thực hiện, mỗi người phụ trách một tình huống: hai lỗi ở mã nguồn (F01, F02), một rủi ro phơi nhiễm ở khâu vận hành (F03) và một fixture cấu hình sai chủ động dựng để kiểm tra chính sách (F04). Mỗi tình huống được trình bày theo mạch **trước khi sửa → sau khi sửa**, gồm cách phát hiện, cách xử lý, nguyên tắc/tiêu chuẩn đối chiếu, pha DevSecOps tương ứng và đoạn mã demo.

Mỗi kịch bản được chạy song song trên hai môi trường để đối chiếu: bản **trước khi sửa** là container chạy image gốc (127.0.0.1:3100), còn bản **sau khi sửa** là bản triển khai đã qua reverse proxy (127.0.0.1:3102). Bên trong container, cổng 3000 là cổng mặc định của Juice Shop; bản đã xử lý vẫn lắng nghe cổng này nhưng không phơi ra host, chỉ reverse proxy (cổng 3102) truy cập được qua mạng nội bộ. Cổng 3100 dành cho container đối chứng chạy image gốc để so sánh. Juice Shop là ứng dụng cố ý chứa lỗ hổng phục vụ huấn luyện; các phát hiện ở đây minh họa quy trình, không nhằm đánh giá dự án gốc.

Bảng 2. Phân công thành viên

Thành viên	Lỗi phụ trách	Nhóm lỗi	Pha DevSecOps
Nguyễn Toàn Thắng — B26CHAT039	F01 SQL injection tại đăng nhập	Mã nguồn	Code, Test
Phạm Thị Minh Thúy — B26CHAT042	F02 Poison Null Byte (lộ tệp sao lưu)	Mã nguồn	Code, Test
Đỗ Đức Quốc Anh — B26CHAT001	F03 Lộ endpoint /metrics không xác thực	Triển khai/vận hành	Deploy, Operate
Hồ Anh Vũ — B26CHAT048	F04 Sai cấu hình an toàn của container	Triển khai (build)	Build, Release

2. Ứng dụng Juice Shop và dữ liệu cần bảo vệ

Juice Shop là một ứng dụng web thương mại điện tử mô phỏng (bán nước ép) do OWASP phát hành nhằm huấn luyện an toàn ứng dụng. Về kiến trúc, đây là một Single Page Application gồm các thành phần sau:

Bảng 3. Thành phần của ứng dụng Juice Shop

Thành phần	Công nghệ	Vai trò
Giao diện người dùng	Angular (SPA)	Hiển thị, gọi REST API
Máy chủ ứng dụng	Node.js + Express	Xử lý REST API và nghiệp vụ
Truy xuất dữ liệu	ORM Sequelize trên SQLite	Lưu Users, Baskets, Cards…
Xác thực phiên	JWT (RS256), thư viện jsonwebtoken	Cấp và kiểm tra token đăng nhập
Giám sát	prom-client, endpoint `/metrics`	Xuất chỉ số dạng Prometheus
Đóng gói	Docker image distroless, chạy UID 65532	Triển khai bằng container

Cả bốn lỗi được phân tích đều gắn với những thành phần này: REST API/ORM (F01), route phục vụ tệp tĩnh (F02), endpoint giám sát (F03) cùng cách đóng gói container (F04). Yêu cầu bảo vệ dữ liệu được xác định theo GDPR (nguyên tắc ở Điều 5, nghĩa vụ an toàn ở Điều 32) và PCI-DSS cho dữ liệu thẻ thanh toán. Song song với khung quốc tế, báo cáo đối chiếu pháp luật Việt Nam **hiện hành tại thời điểm lập báo cáo (tháng 10/2026)**: **Luật Bảo vệ dữ liệu cá nhân 91/2025/QH15** và **Nghị định 356/2025/NĐ-CP** (cùng hiệu lực 01/01/2026, thay thế Nghị định 13/2023/NĐ-CP), cùng **Luật An toàn thông tin mạng 2015** (chi tiết ở mục 3.2).

Bảng 4. Dữ liệu cần bảo vệ và cơ sở chính sách

Dữ liệu	Yêu cầu bảo vệ	Cơ sở chính sách
Email, hồ sơ	Chỉ chủ tài khoản/quản trị truy cập	GDPR Điều 5(1)(f), 25(2)
Mật khẩu	Không để lộ; không bị vượt xác thực	GDPR Điều 32(1)(b)
Token phiên (JWT)	Chỉ cấp khi xác thực đúng	GDPR Điều 32(1)(b)
Giỏ hàng	Ràng buộc chủ sở hữu ở mọi truy vấn	GDPR Điều 5(1)(f), 25(2)
Thẻ thanh toán	Không lộ số thẻ; PAN nếu lưu phải ở dạng không đọc được	PCI-DSS 3.4; GDPR Điều 32
Tệp cấu hình, sao lưu phía máy chủ	Không để tải về qua endpoint phục vụ tệp tĩnh	GDPR Điều 32(1)(b) (liên quan F02)

Ý nghĩa cụ thể của từng điều khoản (GDPR, PCI-DSS… và các chuẩn khác) được tóm gọn trong **Phụ lục C**.

2.1. Nghiệp vụ đăng ký và đăng nhập

Người dùng đăng ký qua `POST /api/Users` và đăng nhập qua `POST /rest/user/login`. Route `routes/login.ts` truy vấn bảng Users, nếu khớp thì cấp JWT kèm mã giỏ hàng. Vấn đề nằm ở ranh giới tin cậy: email do client gửi lên đi thẳng vào câu lệnh SQL, và đó chính là gốc của lỗi **F01**.

2.2. Nghiệp vụ tải tệp tài liệu

Ứng dụng cho phép tải tệp tài liệu trong thư mục `ftp/` qua `GET /ftp/:file`, chỉ cho phép đuôi `.md`/`.pdf`. Route `routes/fileServer.ts` xử lý yêu cầu này. Thư mục `ftp/` còn chứa một số tệp **không nên để tải về** (bản sao lưu cấu hình, mã giảm giá…). Nếu bộ lọc đuôi file bị qua mặt, người dùng có thể đọc các tệp nhạy cảm đó — và đây là gốc của lỗi **F02**.

3. Môi trường triển khai và yêu cầu an toàn tối thiểu

Ứng dụng chạy bằng Docker trên Windows (WSL2) và được triển khai qua `docker-compose.yml`. Các yêu cầu an toàn tối thiểu ở đây bám theo CIS Docker Benchmark cùng nguyên tắc giảm bề mặt tấn công.

Để mỗi kết luận truy được về dữ liệu, thực nghiệm cố định các biến sau và chỉ thay đúng phần cần kiểm chứng (F01/F02 giữ nguyên seed/harness, chỉ thay route; F03 giữ logic ứng dụng, chỉ thay cấu hình truy cập; F04 chạy fixture trước/sau độc lập):

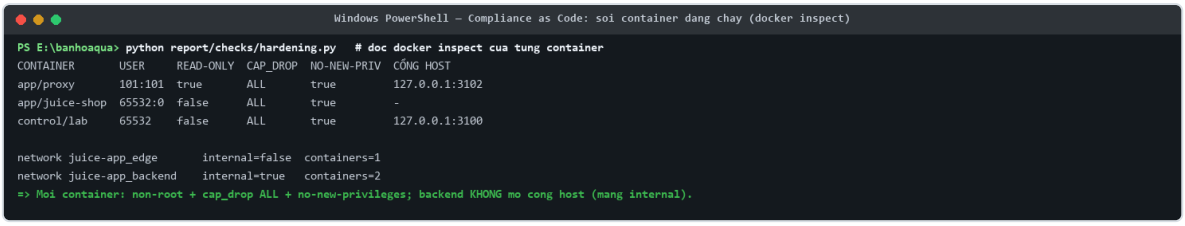
Bảng 5. Phương pháp thực nghiệm

Hạng mục	Giá trị cố định
Phiên bản mã trước khi vá	Bản mã Juice Shop gốc (thư mục `routes/`, `lib/` chưa chỉnh) — dùng để quét Semgrep và chạy ca kiểm thử đỏ
Phiên bản mã sau khi vá	Bản vá của nhóm, đóng gói trong image hotfix triển khai ở cổng 3102
Hai môi trường	Đối chứng `127.0.0.1:3100` (image gốc) ↔ triển khai `127.0.0.1:3102` (proxy + backend mạng internal)
Dữ liệu thử nghiệm	Bộ dữ liệu mẫu Juice Shop tự nạp khi khởi động (người dùng, giỏ hàng, sản phẩm giả lập)
Công cụ (phiên bản)	Semgrep 1.180.0; Gitleaks v8.30.1; Conftest/OPA v0.71.1; node:test + Supertest; Trivy 0.76.1; Dockle 0.4.15; Prometheus 3.15.1; Grafana 13.3.0
Tiêu chí đạt	F01/F02: ca kiểm thử xanh; F03: `/metrics` trả 404 từ ngoài; F04: conftest `exit 0` với bản tốt, `exit 1` với fixture lỗi
Bằng chứng	`report/evidence/*.log` (kèm exit code), kiểm thử, demo (tổng hợp ở mục 4.2)

Bảng 6. Kiểm soát an toàn tối thiểu

Kiểm soát	Cách thực hiện	Tiêu chuẩn
Giảm bề mặt mạng	Chỉ reverse proxy publish cổng, chỉ trên 127.0.0.1; backend ở mạng internal	GDPR Điều 32
Không chạy root	Juice Shop UID 65532, Nginx UID 101	CIS Docker 4.1
Đặc quyền tối thiểu	`cap_drop: ALL`, `no-new-privileges`; hệ thống tệp gốc đặt chế độ chỉ đọc ở các service hỗ trợ	CIS Docker 5.3/5.12
Ghim phiên bản	Luật Rego chặn `:latest`, cảnh báo khi base image chưa ghim digest	CIS Docker Benchmark §4
Lọc đường vào	Nginx chặn /metrics, giới hạn 5 lần đăng nhập/phút	GDPR Điều 32

Những kiểm soát này được **xác minh ngay trên container đang chạy** bằng `docker inspect` (Compliance as Code), cho thấy chúng thực sự có hiệu lực chứ không chỉ nằm trên file cấu hình:



Hình 1. Bằng chứng runtime qua `docker inspect`: cả ba container chạy non-root (USER), `cap_drop: ALL`, `no-new-privileges`; backend không mở cổng host (CỔNG HOST = "-", mạng internal), chỉ proxy mở loopback 127.0.0.1:3102

Vẫn còn một số giới hạn: container đối chứng 3100 chạy image gốc chỉ phục vụ so sánh trước/sau; HTTP nội bộ chưa bật TLS; base image chưa ghim theo digest (luật Rego hiện mới cảnh báo chứ chưa bắt buộc trên image đối chứng).

3.1. Công cụ sử dụng

Bảng 7. Công cụ và vai trò

Công cụ	Loại	Vai trò trong báo cáo
Docker, Docker Compose	Nền tảng container	Triển khai ứng dụng và môi trường kiểm thử
Nginx	Reverse proxy	Chặn /metrics, giới hạn tốc độ đăng nhập (F03)
Semgrep	SAST (phân tích tĩnh)	Đọc mã nguồn (không chạy) để chỉ ra các mẫu lỗi; trong phạm vi báo cáo, xác định chính xác dòng SQL injection của F01 (đăng nhập) và Poison Null Byte của F02 (route tải tệp)
Gitleaks	Quét secret	Tìm khóa/bí mật lộ trong mã nguồn
node:test (Supertest)	Kiểm thử động	Chạy ứng dụng thật, tái hiện lỗi *trước khi vá* (đỏ) và kiểm chứng *sau khi vá* (xanh)
Conftest / OPA (Rego)	Policy as Code	Kiểm tra *văn bản* Dockerfile và cấu hình triển khai (pha Build/Deploy, F03/F04)
Trivy	Quét image (SCA)	Quét CVE dependency + secret + misconfig trên **image đã build** (pha Release); bổ sung mảng SCA mà các công cụ trên không có
Dockle	Lint image (CIS)	Kiểm cấu hình **image sau build** theo CIS Docker — bắt secret bị bake vào layer, thiếu HEALTHCHECK, user hiệu lực (pha Release)
Prometheus + Grafana	Giám sát (Operate)	Thu thập `/metrics` nội bộ, cảnh báo bất thường, dashboard vận hành

Giới hạn của phân tích tĩnh. SAST (Semgrep) chỉ *tìm ra mẫu lỗi* trên mã nguồn và **không chứng minh được hành vi khi chạy**. Chẳng hạn, cảnh báo tại `res.sendFile` (F02) chỉ nêu vị trí nhận tên tệp từ người dùng; phải kiểm thử HTTP mới xác định tệp có thực sự bị trả về

hay không. Những điều đó phải nhờ **kiểm thử động** (node:test) và **rà soát của con người** khẳng định. Chính vì thế, báo cáo kết hợp nhiều lớp theo vòng đời thay vì trông cậy vào một công cụ duy nhất: SAST + kiểm thử động trên **mã nguồn** (pha Code/Test), quét **image** bằng Trivy/Dockle (pha Release), Policy as Code trên văn bản cấu hình (pha Build/Deploy), Compliance as Code trên **container đang chạy** (pha Operate) và **giám sát vận hành** bằng Prometheus/Grafana (pha Operate). Mỗi lớp bắt loại lỗi mà lớp khác không thấy — minh họa ở mục 4.

3.2. Chính sách và tiêu chuẩn đối chiếu

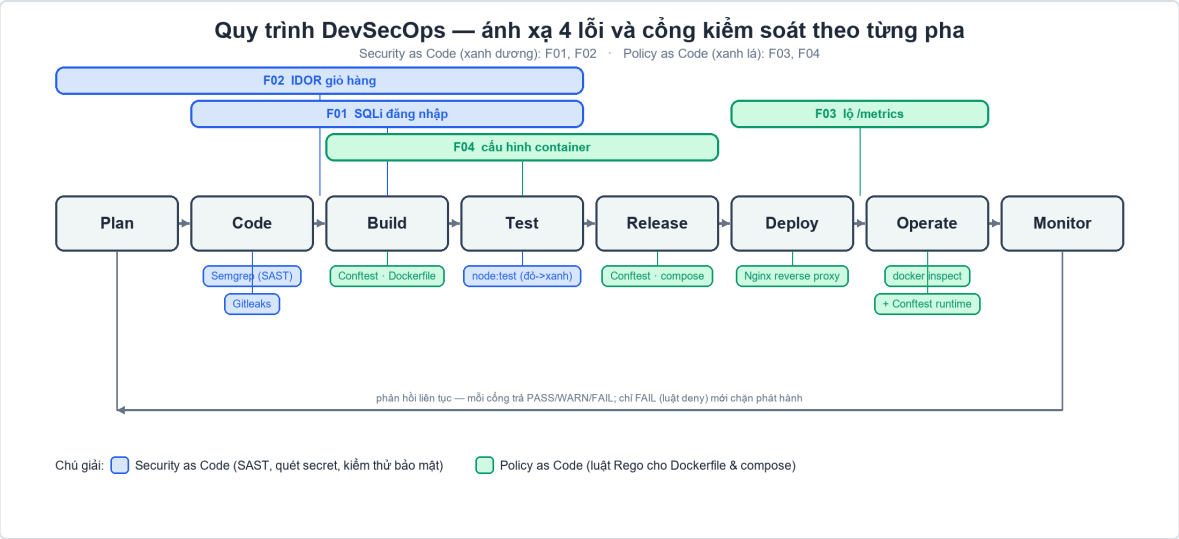
Báo cáo đối chiếu **song song** khung quốc tế và pháp luật Việt Nam về bảo vệ dữ liệu cá nhân; ngưỡng và luật Rego cụ thể do nhóm thiết kế cho môi trường lab. Phần thân chỉ ghi **mã điều khoản — ý nghĩa (yêu cầu gì) của mọi điều khoản tra tại Phụ lục C**.

- **Quốc tế:** GDPR (EU) 2016/679 — Điều 5, 25, 32; PCI-DSS — Yêu cầu 2.2, 3.4, 8.2.1; OWASP Top 10 2021 & CWE — A03/CWE-89 (F01), A01/CWE-158 + CWE-180 (F02), A02/CWE-327 (băm MD5); CIS Docker Benchmark — 4.1, 4.9, 4.10, 5.3, 5.4, 5.8, 5.12 (F04); OCI image-spec, NIST SP 800-190, NIST SSDF (SP 800-218), SLSA (chuỗi cung ứng — chỉ *tham chiếu*, không tuyên bố đạt mức).
- **Việt Nam:** Luật Bảo vệ dữ liệu cá nhân 91/2025/QH15 — Điều 3 (hiệu lực 01/01/2026); Nghị định 356/2025/NĐ-CP — Điều 9, 12 (thay NĐ 13/2023/NĐ-CP từ 01/01/2026); Luật An toàn thông tin mạng 2015 (86/2015/QH13) — Điều 16, 17, 19; TCVN 11930:2017.

Hai lưu ý về hiệu lực: PCI-DSS v3.2.1 đã **nghỉ từ 31/3/2024** (bản hiện hành v4.0.1, số điều đổi, vd 3.4 → 3.5.1) nên chỉ trích như **đối chiếu lịch sử**; Nghị định 13/2023/NĐ-CP đã hết hiệu lực nên không dùng làm khung hiện hành. Với dữ liệu giả lập của ứng dụng huấn luyện, báo cáo dùng cách diễn đạt *"đối chiếu nguyên tắc bảo vệ"* thay cho kết luận vi phạm pháp luật hay chứng nhận tuân thủ.

4. Quy trình DevSecOps và các cổng kiểm soát

Việc phát hiện và chống tái diễn cả bốn lỗi đều được viết thành mã và chạy tự động dưới dạng một chuỗi kiểm soát (pipeline cục bộ) theo mô hình DevSecOps. Hình dưới ánh xạ từng lỗi và cổng kiểm soát vào các pha trong vòng đời DevSecOps (xanh dương là Security as Code cho F01/F02, xanh lá là Policy as Code cho F03/F04):



Hình 2. Quy trình DevSecOps: ánh xạ bốn lỗi F01–F04 và các cổng kiểm soát theo từng pha; Security as Code cho F01/F02, Policy as Code cho F03/F04

Các lớp kiểm soát, xếp theo **pha vòng đời** (mỗi lớp bắt loại lỗi lớp khác không thấy):

- **Security as Code (Code/Test):** quét secret (Gitleaks), SAST (Semgrep) cho F01/F02, và kiểm thử bảo mật `f01-f02.test.ts` (đỏ trước vá, xanh sau vá).
- **Policy as Code (Build/Deploy):** luật Rego cho Dockerfile (F04) và `docker-compose` (F03), chặn ngay cấu hình mở cổng backend hoặc chạy root **trước khi triển khai** — đọc *văn bản* cấu hình.
- **Quét image (Release): Trivy** quét CVE dependency (SCA) và **Dockle** lint cấu hình image theo CIS trên **image đã build** — bắt lỗ hổng thư viện và secret bị bake vào layer mà quét văn bản không thấy (mục 4.1).
- **Compliance as Code (Operate):** kiểm **container đang chạy** bằng `docker inspect` + luật Rego runtime, xác nhận non-root, drop capability, chỉ proxy mở cổng loopback. Đặt container đối chứng (trước vá) cạnh bản triển khai (sau vá) tạo đúng cặp ca *lỗi cố ý / đạt*.
- **Giám sát vận hành (Operate): Prometheus** scrape `/metrics` nội bộ, **Grafana** dashboard, và **cảnh báo** khi có bất thường (vd đột biến 4XX do dò đăng nhập) — mục 4.3.

Việc kiểm chứng diễn ra trên **hai môi trường chạy song song**: container đối chứng *trước vá* (image gốc, `127.0.0.1:3100`) và bản triển khai *sau vá* (reverse proxy kèm backend ở mạng internal, `127.0.0.1:3102`). Cả hai đều đang hoạt động:



Hình 3. Hai môi trường: đối chứng trước vá (3100) và triển khai sau vá qua proxy (3102); backend chạy ở mạng internal, không mở cổng host

Toàn bộ kiểm soát này được viết thành mã nên chạy lặp lại được. **Hiện chúng chạy cục bộ**, chưa được nối thành *required check* bắt buộc trong CI; báo cáo vì vậy phân biệt rõ giữa "đã chạy và có mã thoát" với "đã chặn phát hành thật". Trong số đó, Conftest trả **mã thoát khác 0 khi vi phạm luật** `deny` (đã kiểm chứng: `exit 1` trên `Dockerfile.vulnerable`, mục 4.1) nên đủ điều kiện làm cổng chặn; còn `semgrep scan` **mặc định vẫn trả** `exit 0` **dù có cảnh báo** (phải thêm `--error` mới chặn), nên trong báo cáo này Semgrep đóng vai trò *phát hiện* chứ không phải cổng chặn. Mức **WARN (luật** `warn`**)** chỉ cảnh báo: trong `compose.rego`, `cap_drop` và `read_only` xếp WARN, còn mở cổng backend, bind ngoài loopback, privileged, mount docker.sock và secret nội tuyến đều ở mức FAIL.

Bảng 8. Các cổng kiểm soát và vai trò đã kiểm chứng

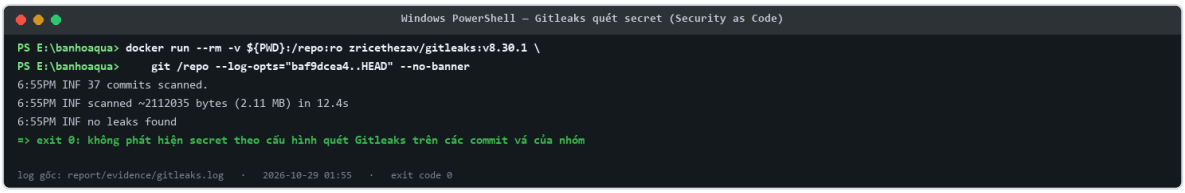
Cổng kiểm soát	Công cụ	Lỗi được kiểm soát	Vai trò cổng (theo mã thoát đã kiểm chứng)
Quét secret	Gitleaks	khóa/bí mật lộ trong mã	exit≠0 khi phát hiện secret; lần chạy cho exit 0 (không có)
SAST	Semgrep	F01, F02	*phát hiện*; `scan` mặc định exit 0 dù có cảnh báo (cần `--error` để chặn)
Kiểm thử bảo mật	node:test (Supertest)	F01, F02 (đỏ→xanh)	exit≠0 khi còn ca đỏ (trước vá), exit 0 sau vá
Policy as Code — build	Conftest/OPA (Rego)	F03, F04 (Dockerfile, compose)	exit≠0 khi vi phạm `deny` — **đã kiểm chứng** (exit 1 trên fixture lỗi); `warn` chỉ cảnh báo
Quét image — CVE (SCA)	Trivy	lỗ hổng dependency + secret trong image	*phát hiện* — 189 CVE (12 CRITICAL, 56 HIGH); `--exit-code 1` đủ làm cổng
Quét image — CIS	Dockle	cấu hình image sau build (secret, HEALTHCHECK, user)	exit≠0 khi có FATAL; bắt secret bake vào layer mà Conftest không thấy
Compliance as Code — runtime	Conftest + `docker inspect`	F03, F04 (container đang chạy)	đối chứng runtime, không dùng để chặn phát hành
Giám sát vận hành	Prometheus + Grafana	bất thường runtime (đột biến 4XX)	thu thập + **cảnh báo fire thật** khi mô phỏng dò đăng nhập; không dùng để chặn phát hành

Quy trình được tổ chức thành các bước nối tiếp theo vòng đời DevSecOps, mỗi bước gắn với một nhóm công cụ kiểm soát cụ thể và một pha:

Bảng 9. Các bước của quy trình DevSecOps áp dụng (theo pha)

Bước	Pha	Hoạt động	Công cụ
1. Thiết lập môi trường	—	Dựng hai môi trường trước/sau vá bằng Docker	Docker Compose, Node.js
2. Policy as Code	Build/Deploy	Kiểm chính sách Dockerfile và compose trước khi triển khai	Conftest/OPA (Rego)
3. Phát hiện điểm yếu mã nguồn	Code	Quét tĩnh mã nguồn, ánh xạ sang điều khoản	Semgrep
4. Test-Driven Security	Test	Viết kiểm thử đỏ trước vá, xanh sau vá	node:test (Supertest)
5. Vá ứng dụng	Code	Sửa mã theo kết quả kiểm thử	TypeScript / Express
6. Quét image	Release	Quét CVE/secret (SCA) và CIS trên image đã build	Trivy, Dockle
7. Compliance as Code	Operate	Kiểm cấu hình container tại thời điểm chạy	Conftest + `docker inspect`
8. Giám sát vận hành	Operate	Thu thập `/metrics` nội bộ, cảnh báo bất thường, dashboard	Prometheus, Grafana
9. Kiểm chứng và bàn giao	—	Chạy lại toàn bộ cổng kiểm soát; quét secret	Conftest, Gitleaks

Kết quả Gitleaks và Conftest được lưu trong `gitleaks.log` và `conftest-verify.log` (kèm thời điểm, phiên bản công cụ và mã thoát). Các hình dưới được dựng lại từ các log đó; dòng kết luận PASS/chặn tính theo **mã thoát** của lệnh, và mỗi hình ghi kèm thời điểm và exit code ở chân hình.



Hình 4. Gitleaks quét các commit vá của nhóm: `exit 0` — không phát hiện secret theo cấu hình quét



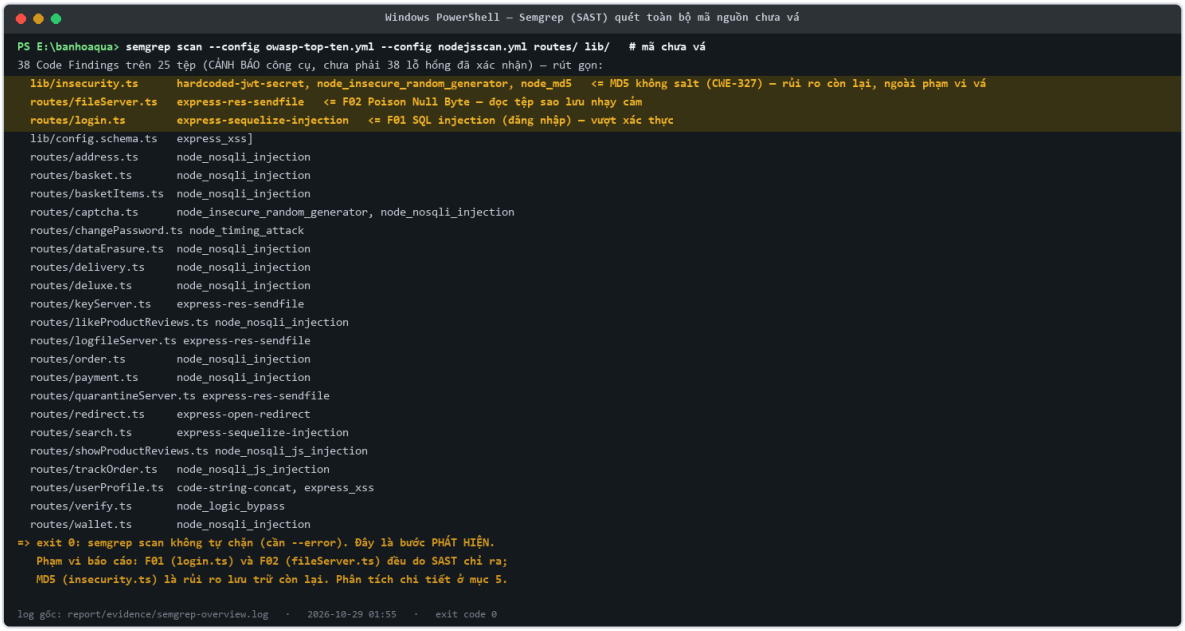
Hình 5. Conftest verify: 19 unit test của bộ luật Rego (13 ca Dockerfile + 6 ca compose), `exit 0` — đều đạt

Mỗi lỗi đi kèm một script demo (`demos/tv1`…`tv4`) tái hiện lỗi trước khi vá và kiểm chứng sau khi vá; các script này được trình bày trong từng mục tương ứng ở phần 5.

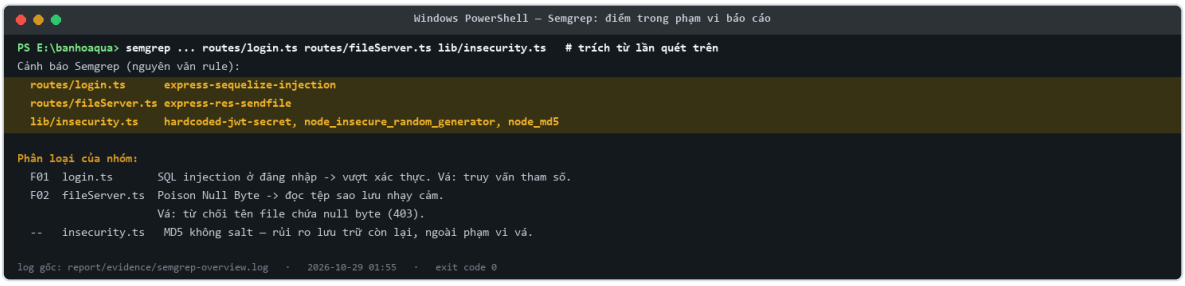
4.1. Quét và phát hiện lỗi — kết quả thực tế

Các cổng quét tự động được chạy trên mã nguồn và cấu hình *chưa vá* để lộ ra điểm yếu trước khi sửa.

SAST (Semgrep). Quét toàn bộ `routes/` và `lib/` (mã *chưa vá*) cho ra **38 cảnh báo** trên 25 tệp (SQL/NoSQL injection, XSS, băm yếu, JWT secret cố định…). Đây là **cảnh báo công cụ, chưa phải 38 lỗ hổng đã xác nhận**; mỗi cảnh báo là một điểm cần rà soát thêm. Thuộc phạm vi báo cáo, phân tích tĩnh chỉ ra ba điểm: **F01** tại `login.ts` (rule `express-sequelize-injection` — SQL injection ở đăng nhập), **F02** tại `fileServer.ts` (rule `express-res-sendfile` — Poison Null Byte, lộ tệp sao lưu), cùng **băm mật khẩu MD5 không salt** tại `lib/insecurity.ts` (rule `node_md5`, CWE-327 — rủi ro lưu trữ còn lại, ngoài phạm vi vá).



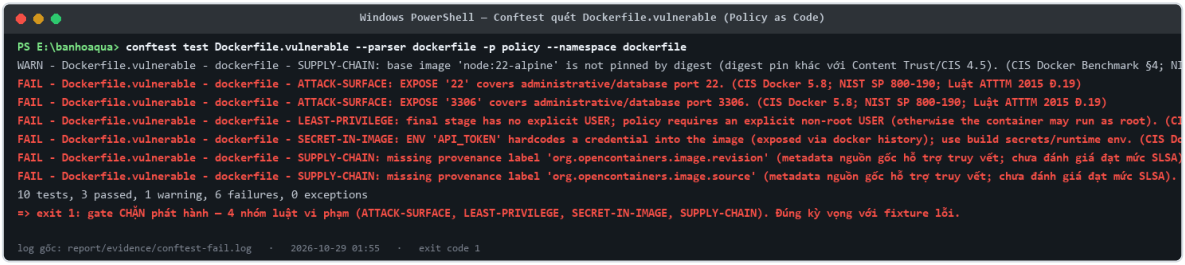
Hình 6. Semgrep quét toàn bộ mã nguồn chưa vá: 38 cảnh báo trên 25 tệp (cảnh báo công cụ, chưa phải lỗ hổng đã xác nhận); tô vàng F01 (`login.ts`), F02 (`fileServer.ts`) và băm MD5 (`insecurity.ts`); lệnh `scan` trả `exit 0` nên chỉ là bước phát hiện, không tự chặn



Hình 7. Ba điểm Semgrep xác định trong phạm vi: F01 SQL injection ở đăng nhập (`login.ts`), F02 Poison Null Byte (`fileServer.ts`) và băm MD5 không salt (`lib/insecurity.ts`); kèm phân loại của nhóm

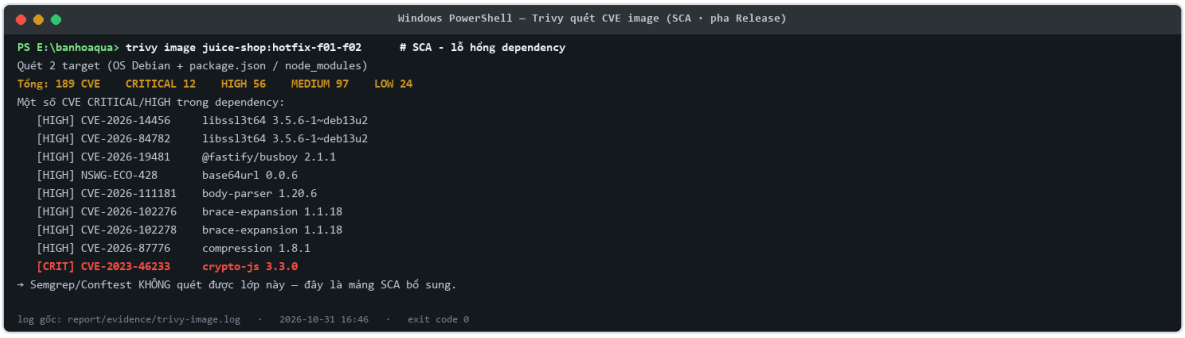
Policy as Code (Conftest). Quét `Dockerfile.vulnerable` cho `exit 1` với **6 thông báo vi phạm thuộc 4 nhóm luật** `deny` — mã thoát này đủ điều kiện làm cổng chặn (chi tiết F04 ở

mục 5.4):



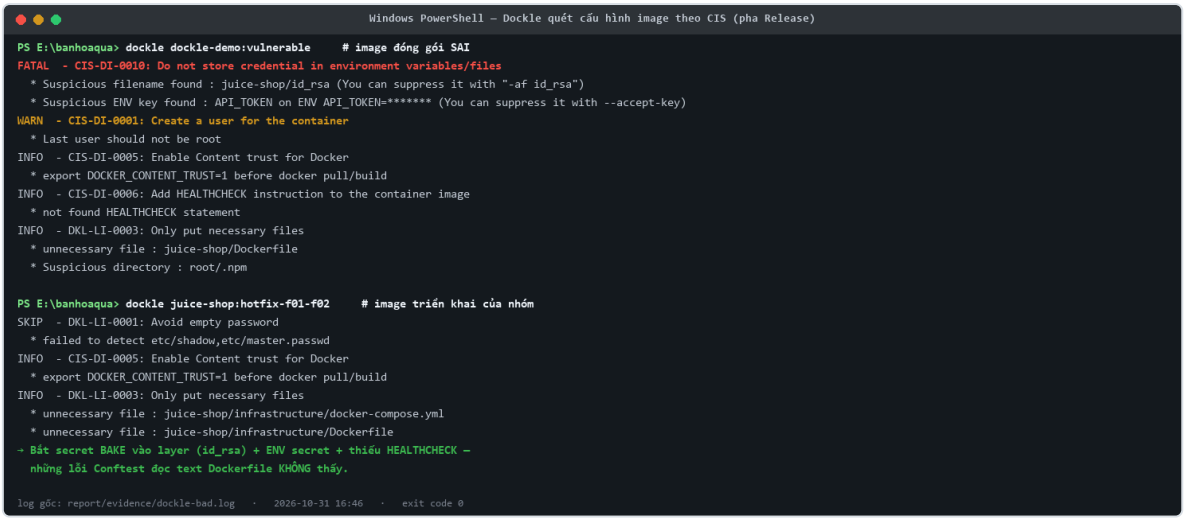
Hình 8. Conftest quét Dockerfile.vulnerable: `exit 1`, 6 thông báo vi phạm thuộc 4 nhóm luật `deny`; mã thoát đủ để làm cổng chặn

Quét image — CVE/SCA (Trivy, pha Release). Trên image triển khai `juice-shop:hotfix-f01-f02`, Trivy quét lỗ hổng *dependency*: **189 CVE** (12 CRITICAL, 56 HIGH, 97 MEDIUM, 24 LOW) — gồm các thư viện cũ như `crypto-js` (CVE-2023-46233, CRITICAL), `jws`, `body-parser`, `brace-expansion`. Đây là **mảng SCA** mà Semgrep (quét mã nhóm tự viết) và Conftest (quét cấu hình) **không phủ**; lỗ hổng thư viện là rủi ro chuỗi cung ứng cần quét ở pha Release (Hình 9).



Hình 9. Trivy quét CVE dependency của image hotfix: 189 CVE — 12 CRITICAL, 56 HIGH; SCA là lớp mà SAST/Policy-as-Code không phủ

Quét image — CIS (Dockle, pha Release). Dockle đọc *image đã build* (không phải văn bản Dockerfile) nên bắt được lỗi chỉ hiện ở thành phẩm. Trên một image đóng gói sai, Dockle báo **FATAL** vì **secret bị bake vào layer** (`id_rsa`) và secret trong `ENV` — thứ Conftest đọc văn bản Dockerfile **không thấy** — cùng cảnh báo chạy root và thiếu HEALTHCHECK; image triển khai của nhóm thì sạch (chỉ còn INFO). Dockle vì vậy bổ sung cho Conftest chứ không trùng: Conftest kiểm *công thức* trước build, Dockle kiểm *thành phẩm* sau build (Hình 10).



Hình 10. Dockle quét cấu hình image theo CIS: image sai bị FATAL vì secret bake vào layer (id_rsa) + ENV; image triển khai của nhóm sạch

4.2. Bảng kết quả hợp nhất

Bảng dưới liên kết từng tình huống với kết quả và **tệp bằng chứng** tương ứng (log gốc trong `report/evidence/`, kiểm thử, demo).

Bảng 10. Kết quả hợp nhất bốn tình huống (điều kiện → kỳ vọng → trước/sau vá → bằng chứng → rủi ro còn lại)

Mã	Điều kiện kiểm thử	Kết quả mong đợi	Trước vá	Sau vá	Bằng chứng	Rủi ro còn lại
F01	Đăng nhập bằng payload SQL + mật khẩu sai	Từ chối (401)	Cấp token, vượt xác thực	401; đăng nhập hợp lệ vẫn 200	`f01-f02.test.ts`; demo `tv1`; `semgrep-overview.log`	Băm MD5 không salt (ngoài phạm vi F01)
F02	`GET /ftp/<tên>%2500.md` (poison null byte)	Từ chối tệp đuôi bị cấm	Đọc được `package.json.bak`, `coupons_2013.md.bak`	403; tệp `.md` hợp lệ vẫn tải	`f01-f02.test.ts`; demo `tv2`; `tv2-f02.log`	Mới thử 2 payload null byte; chưa đánh giá toàn bộ chuẩn hóa đường dẫn và chính sách công bố tệp
F03	`GET /metrics` từ host	Chặn truy cập từ ngoài	200, lộ chỉ số vận hành	404 qua proxy; backend không mở cổng host	demo `tv3`; `env-*.log`; `hardening`	Giám sát nội bộ vẫn đọc được (đúng thiết kế)
F04	Conftest trên fixture lỗi / bản vá / Dockerfile gốc	Chặn fixture lỗi, đạt	Fixture: `exit 1`, 6 vi phạm / 4 nhóm	Bản vá & gốc: đạt; `verify` 19/19	`conftest-fail.log`; `conftest-verify.log`; demo `tv4`	Đánh giá giới hạn ở các luật đã định

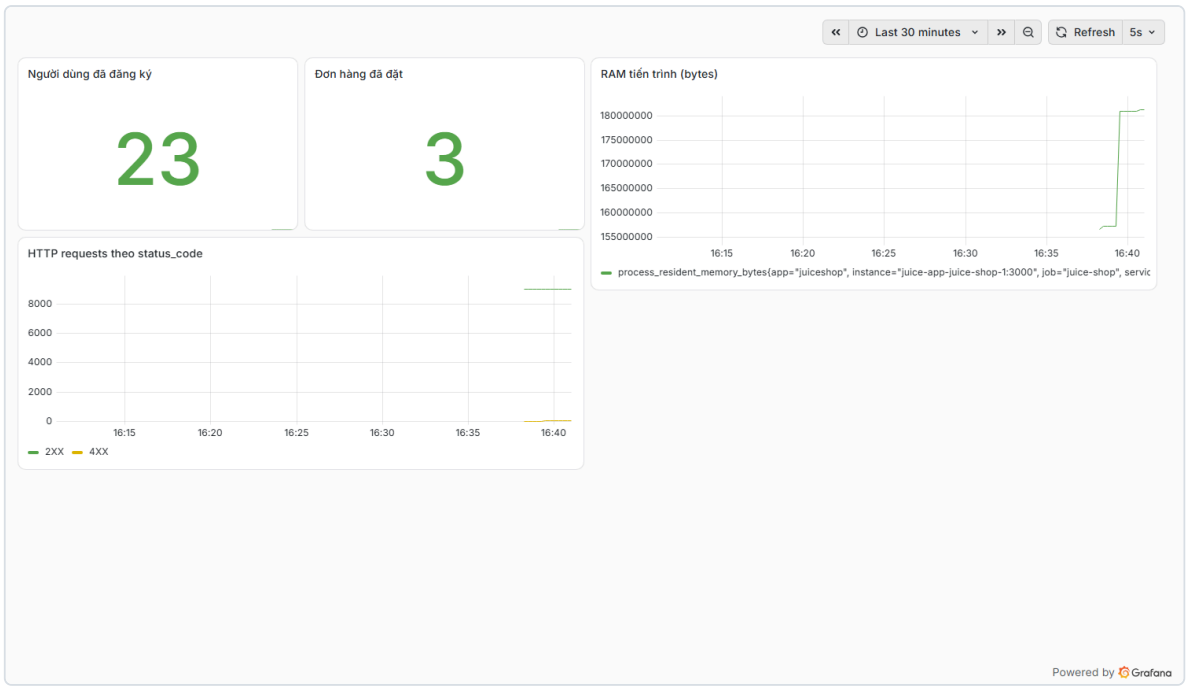
Mã	Điều kiện kiểm thử	Kết quả mong đợi	Trước vá	Sau vá	Bằng chứng	Rủi ro còn lại
		bản tốt				nghĩa; chưa ghim digest (mới cảnh báo)

Các chỉ số ở F03 là số liệu tổng hợp, chưa phải dữ liệu cá nhân định danh; F04 là fixture cố ý lỗi, không phải lỗi image gốc. Những điểm "ngoài phạm vi" (MD5, TLS, digest) được giữ lại ở cột *rủi ro còn lại* thay vì coi là đã xử lý.

4.3. Giám sát khi vận hành (pha Operate)

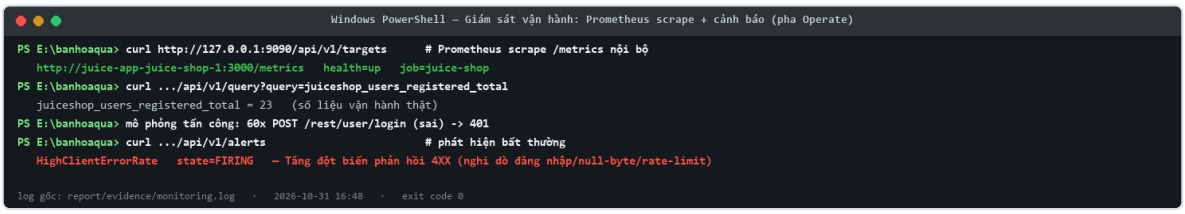
Các lớp trên dừng ở "trước khi chạy" và "cấu hình lúc chạy". Khi ứng dụng **đang chạy**, nhóm bổ sung một lớp **quan sát + phát hiện** nhẹ: **Prometheus** scrape endpoint `/metrics` của ứng dụng qua **mạng nội bộ** (app vẫn không mở cổng host — đúng nguyên tắc F03), **Grafana** làm portal hiển thị, và một **luật cảnh báo** để phát hiện bất thường.

Chính `/metrics` mà F03 chặn ra ngoài lại là *nguồn cấp dữ liệu giám sát nội bộ*: nó xuất số người dùng, số đơn hàng, thời gian xử lý HTTP và số phản hồi theo mã trạng thái. Prometheus scrape endpoint này (target **UP**), Grafana vẽ thành dashboard vận hành (Hình 11).



Hình 11. Dashboard Grafana (portal giám sát) với số liệu thật từ ứng dụng đang chạy: người dùng đã đăng ký, đơn hàng, HTTP theo status_code, RAM tiến trình

Phần **phát hiện**: luật cảnh báo `HighClientErrorRate` theo dõi đột biến phản hồi **4XX** — dấu hiệu dò mật khẩu (F01), null-byte (F02) hoặc bị rate-limit (F03). Khi nhóm mô phỏng tấn công (60 lần đăng nhập sai liên tiếp), số 4XX vọt lên và cảnh báo **chuyển trạng thái FIRING**, minh họa việc phát hiện tấn công mà bản thân mã nguồn không tự biết (Hình 12). Đây là biện pháp *giám sát liên tục* mà NĐ 356/2025/NĐ-CP nêu ở khâu vận hành.



Hình 12. Giám sát vận hành: Prometheus scrape /metrics (target UP), truy vấn số liệu thật, và cảnh báo HighClientErrorRate chuyển FIRING khi mô phỏng dò đăng nhập

4.4. Lệnh quét (tái lập)

Các lệnh **quét** của từng cổng kiểm soát được gom ở đây để tái lập, **tách riêng** với các script demo tấn công/khắc phục (`tv1`–`tv4` ở phần 5). Mỗi lệnh chạy ra log gốc kèm mã thoát trong `report/evidence/`.

```
# (Code)         SAST — quét mã nguồn
semgrep scan --config auto routes/ lib/
# (Code)         quét secret trong lịch sử commit
gitleaks git /repo --log-opts=<range> --no-banner
# (Build/Deploy) Policy as Code — quét VĂN BẢN Dockerfile + compose
conftest test Dockerfile.vulnerable -p policy --namespace dockerfile
conftest test docker-compose.yml    -p policy --namespace compose
```

```
conftest verify -p policy                                    # unit test bộ luật Rego
# (Release)      quét IMAGE đã build
trivy  image --scanners vuln juice-shop:hotfix-f01-f02       # CVE / SCA
dockle juice-shop:hotfix-f01-f02                             # CIS image (secret, HEALTHCHECK, user)
# (Operate)      Compliance as Code + giám sát
docker inspect <container>                                   # -> đưa JSON vào Conftest runtime
curl http://127.0.0.1:9090/api/v1/targets                   # Prometheus scrape /metrics
curl http://127.0.0.1:9090/api/v1/alerts                    # trạng thái cảnh báo
```

Ảnh được dựng lại từ chính các lệnh này: `report/figures/capture_runs.py` (Semgrep/Gitleaks/Conftest/docker inspect) và `report/figures/capture_scans.py` (Trivy/Dockle/Prometheus).

5. Bốn lỗi an toàn: phát hiện và xử lý

5.1. F01 — SQL injection tại đăng nhập

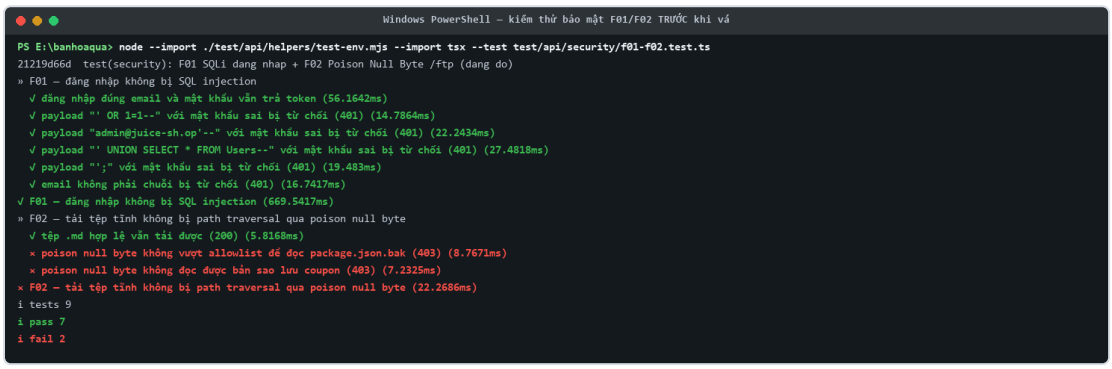
Trước khi sửa

Route `routes/login.ts` ghép thẳng email vào câu SQL:

```
models.sequelize.query(`SELECT * FROM Users WHERE email = '${req.body.email || ''}'
  AND password = '${security.hash(req.body.password || '')}' AND deletedAt IS NULL`, { model:
UserModel, plain: true })
```

Với email `' OR 1=1--`, mệnh đề WHERE luôn đúng còn phần kiểm tra mật khẩu bị đẩy thành chú thích, nên máy chủ cấp token của tài khoản đầu tiên (tài khoản quản trị) mà không cần mật khẩu. Đây chính là CWE-89, OWASP A03:2021.

Phát hiện. Semgrep (luật `express-sequelize-injection`) chỉ đúng dòng lỗi (xem hình quét tổng ở mục 4.1). Kiểm thử `test/api/security/f01-f02.test.ts` gửi payload kèm mật khẩu sai và yêu cầu phản hồi 401; trên mã chưa sửa, test báo đỏ (Hình 13). Demo `demos/tv1_f01_sqli.py` xác nhận token vẫn được cấp (Hình 14).



Hình 13. Kiểm thử bảo mật F01/F02 trên mã chưa sửa: các ca kiểm thử đỏ ứng với lỗ hổng còn tồn tại

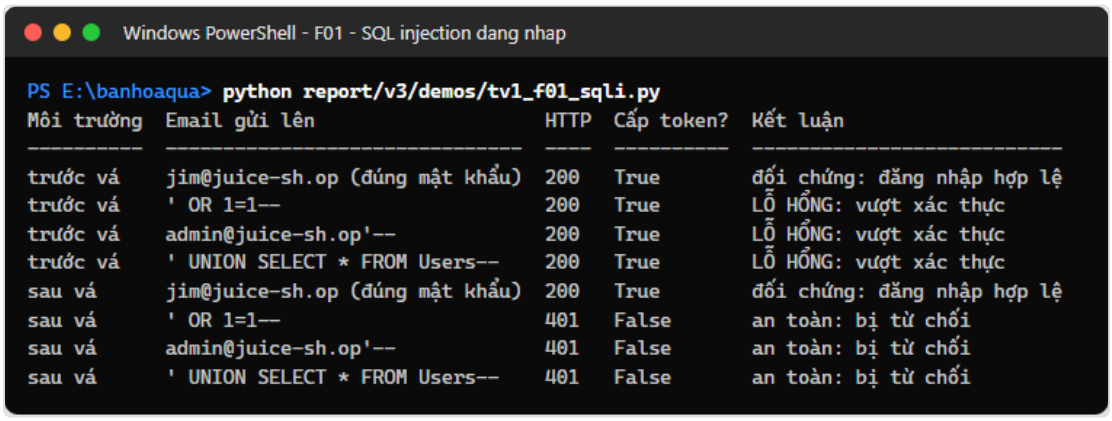
Sau khi sửa

Dùng truy vấn tham số (bind parameter):

```
const email = typeof req.body.email === 'string' ? req.body.email : ''
const password = typeof req.body.password === 'string' ? req.body.password : ''
models.sequelize.query('SELECT * FROM Users WHERE email = $email AND password = $password AND
deletedAt IS NULL',
  { bind: { email, password: security.hash(password) }, model: UserModel, plain: true })
```

Câu lệnh được biên dịch trước và giá trị truyền riêng, nên mọi ký tự SQL trong email giờ chỉ còn được hiểu là dữ liệu. Sau khi sửa, kiểm thử bảo mật xanh toàn bộ (Hình 16); demo cũng cho thấy mọi payload đều bị từ chối 401 trong khi đăng nhập hợp lệ vẫn hoạt động bình thường (Hình 14).

Bản vá vẫn còn một giới hạn: nó chỉ xử lý injection, trong khi cùng dòng code đó vẫn gọi `security.hash` của Juice Shop, vốn băm mật khẩu bằng **MD5 không salt** (`lib/insecurity.ts`, CWE-327 / OWASP A02:2021). Đây là lỗ hổng lưu trữ mật khẩu nằm ngoài phạm vi F01, vì F01 chỉ chặn việc vượt xác thực; khi triển khai thật, cần thay nó bằng bcrypt hoặc argon2 có salt.



Hình 14. Demo F01: cùng payload, trước khi sửa được cấp token, sau khi sửa bị từ chối

Nguyên tắc/tiêu chuẩn đối chiếu: GDPR Điều 32(1)(b) (bảo mật khi xử lý dữ liệu); Luật 91/2025/QH15 Điều 3 và NĐ 356/2025/NĐ-CP (biện pháp kỹ thuật bảo vệ dữ liệu cá nhân); Luật ATTTM 2015 Điều 19. **Pha DevSecOps:** Code → Test. **Kiểm soát (Security as Code):** phát hiện bằng SAST ở pha Code, chặn tái diễn bằng kiểm thử bảo mật tự động ở pha Test.

5.2. F02 — Poison Null Byte làm lộ tệp sao lưu

Trước khi sửa

Route `routes/fileServer.ts` phục vụ tệp trong thư mục `ftp/` qua `GET /ftp/:file`. Ứng dụng kiểm tra hậu tố `.md`/`.pdf` (ngoài ra có ngoại lệ cho `incident-support.kdbx`), sau đó mới phục vụ:

```
if (file && (endsWithAllowlistedFileType(file) || (file === 'incident-support.kdbx'))) {
  file = security.cutOffPoisonNullByte(file)   // cắt tên tại chuỗi literal "%00" rồi mới phục vụ
```

```
  res.sendFile(path.resolve('ftp/', file))
}
```

Điểm yếu nằm ở **thứ tự xử lý**: tên tệp được *kiểm tra* khác với tên *thực sự dùng để đọc*. Với yêu cầu `/ftp/package.json.bak%2500.md`, quá trình giải mã URL tạo tham số `package.json.bak%00.md`; tham số này vượt qua kiểm tra hậu tố `.md`, rồi hàm `cutOffPoisonNullByte` cắt tại chuỗi `%00` thành `package.json.bak` và chuyển cho `res.sendFile`. Nhờ đó người dùng **đọc được các tệp sao lưu phía máy chủ** mà bộ lọc đuôi lẽ ra chặn — ví dụ `package.json.bak` (thông tin cấu hình, phiên bản dependency) hay `coupons_2013.md.bak` (mã giảm giá). Đây là lỗi **Poison Null Byte** (CWE-158; CWE-180 kiểm tra trước khi chuẩn hóa; OWASP A01:2021 Broken Access Control). Tình huống thực nghiệm chỉ chứng minh đọc được tệp *bên trong* thư mục `ftp/`; chưa chứng minh truy cập ra ngoài thư mục này (`/` bị chặn riêng).

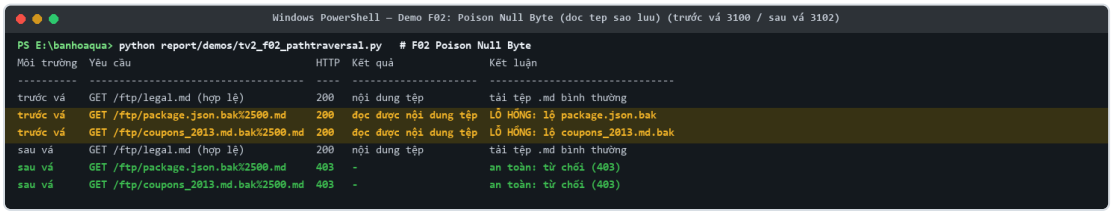
Phát hiện. Semgrep (luật `express-res-sendfile`) chỉ ra lời gọi `res.sendFile` nhận tên tệp từ đầu vào người dùng — F02 là một trong các cảnh báo SAST được xác nhận (xem hình quét ở mục 4.1). Vì cảnh báo chỉ nêu vị trí, nhóm phân tích luồng xử lý và dùng **kiểm thử HTTP** để xác nhận việc vượt bộ lọc: kiểm thử `test/api/security/f01-f02.test.ts` gửi `GET /ftp/package.json.bak%2500.md` và yêu cầu phản hồi 403; trên mã chưa sửa, ca này báo đỏ (Hình 13). Demo `demos/tv2_f02_pathtraversal.py` cho thấy cùng payload đọc được nội dung tệp ở bản chưa vá (Hình 15).

Sau khi sửa

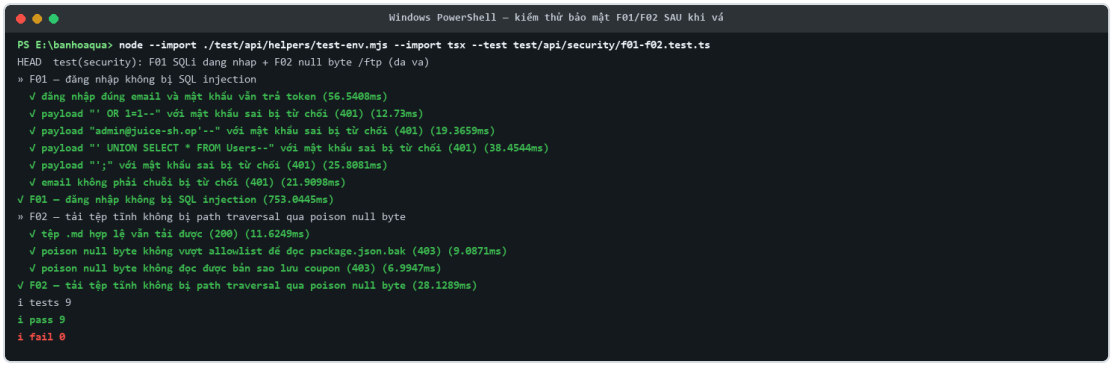
Từ chối hẳn tên tệp chứa null byte (`%00` dạng chuỗi hoặc ký tự NUL thực `\u0000`) **trước** bước kiểm tra loại và phục vụ, thay vì cắt bỏ rồi gửi:

```
if (file && (file.includes('%00') || file.includes('\u0000'))) {
  res.status(403)
  next(new Error('File names cannot contain null bytes!'))
  return
}
```

Sau khi sửa, hai payload `...%2500.md` bị từ chối **403**, trong khi tải tệp hợp lệ (`legal.md`) vẫn trả về bình thường (Hình 15); kiểm thử bảo mật khi đó xanh toàn bộ (Hình 16). Kết quả xác nhận hiệu quả của bản vá *với các payload đã thử*, chưa đại diện cho đánh giá mọi biến thể chuẩn hóa đường dẫn hay toàn bộ chính sách công bố tệp.



Hình 15. Demo F02: trước vá payload null byte đọc được `package.json.bak` và `coupons_2013.md.bak`; sau vá trả 403, tệp `.md` hợp lệ vẫn tải được



Hình 16. Kiểm thử bảo mật F01/F02 sau khi sửa: toàn bộ 9 ca đạt

Nguyên tắc/tiêu chuẩn đối chiếu: GDPR Điều 5(1)(f), 32(1)(b) (bảo mật, toàn vẹn khi xử lý); Luật 91/2025/QH15 Điều 3 và NĐ 356/2025/NĐ-CP; Luật ATTTM 2015 Điều 17 (không để lộ dữ liệu cho bên thứ ba). *Tệp bị lộ ở đây là cấu hình và mã giảm giá, không phải dữ liệu thẻ, nên không viện dẫn PCI-DSS.* **Pha DevSecOps:** Code → Test. **Kiểm soát (Security as Code):** phát hiện bằng SAST ở pha Code, chặn tái diễn bằng kiểm thử bảo mật tự động ở pha Test.

5.3. F03 — Lộ endpoint /metrics không xác thực

Trước khi sửa

Juice Shop mở sẵn `/metrics` (Prometheus) mà không yêu cầu đăng nhập, xuất ra các chỉ số như số người dùng, số đơn hàng và tổng số dư ví. Khi container publish thẳng cổng ứng dụng, **mọi chủ thể có đường truy cập tới host** đều gọi được endpoint này; trong lab cổng chỉ bind loopback (`127.0.0.1`) nên phạm vi là *truy cập từ host*, chưa phải phơi ra Internet. Bản thân các chỉ số là **số liệu tổng hợp vận hành**, chưa đủ để coi là dữ liệu cá nhân định danh; tuy nhiên để một endpoint nội bộ không xác thực truy cập được vẫn vi phạm nguyên tắc *giảm bề mặt tấn công và tối thiểu phơi bày*. Đây là một rủi ro thuộc khâu **triển khai/vận hành**.

Phát hiện. Demo `demos/tv3_f03_metrics.py` gọi `/metrics` từ host; trước khi sửa, endpoint trả về 200 kèm hàng trăm dòng số liệu (Hình 17).

Sau khi sửa

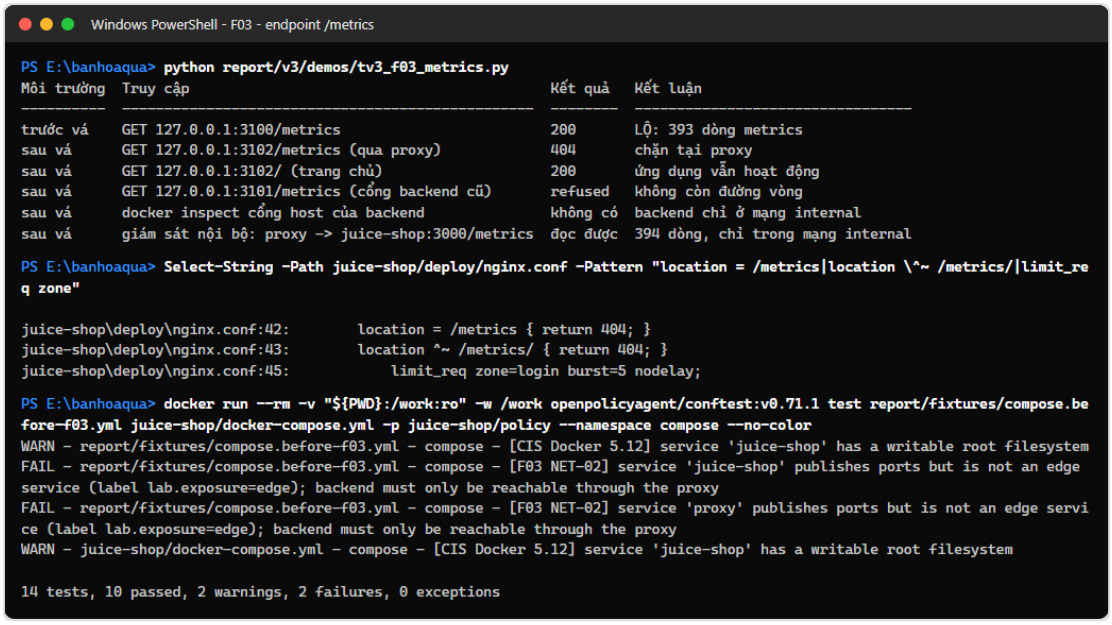
Hướng xử lý là bỏ cổng host của backend và chỉ để reverse proxy mở cổng loopback; tại proxy, `/metrics` bị chặn và tốc độ đăng nhập bị giới hạn:

```
location = /metrics { return 404; }
location = /rest/user/login { limit_req zone=login burst=5 nodelay; limit_req_status 429;
proxy_pass http://juice-shop:3000; }
```

```
juice-shop:
  networks: [backend]            # không còn khóa ports -> không truy cập trực tiếp từ host
```

```
networks:
  backend: { internal: true }
```

Sau khi sửa, `/metrics` đi qua proxy trả về 404 và cổng backend cũ bị từ chối kết nối **từ host**. Đây là chặn ở lối vào từ bên ngoài chứ không phải khóa tuyệt đối: trong mạng internal, hệ thống giám sát vẫn đọc được metrics đúng mục đích (Hình 17).



Hình 17. Demo F03: /metrics lộ khi truy cập trực tiếp, bị chặn sau khi sửa

Nguyên tắc/tiêu chuẩn đối chiếu: GDPR Điều 25(2), 32(1)(b); Luật 91/2025/QH15 Điều 3 và NĐ 356/2025/NĐ-CP; Luật ATTTM 2015 Điều 17, 19. **Pha DevSecOps:** Deploy → Operate. **Kiểm soát (Policy as Code):** luật `compose.rego` chặn mọi cấu hình để backend mở cổng host (mục 4).

5.4. F04 — Sai cấu hình an toàn của container

Khác ba lỗi trên, F04 **không phải lỗi của image Juice Shop gốc** (image distroless gốc đã chạy non-root và đạt bộ luật). Đây là một **fixture cấu hình sai chủ động dựng** (`Dockerfile.vulnerable`) để kiểm tra *năng lực phát hiện* của bộ policy, sử dụng cả đầu vào "xấu" (chứng minh luật bắt được) và đầu vào "tốt" (chứng minh luật không báo nhầm).

Trước khi sửa

Một image đóng gói Juice Shop có thể vi phạm chính sách an toàn tối thiểu mà vẫn lọt qua nếu kiểm tra đặt sai chỗ. `Dockerfile.vulnerable` dưới đây minh họa bốn vi phạm như vậy:

```
FROM node:22-alpine
COPY . .
ENV API_TOKEN=supersecret123  # nhúng bí mật vào image (lộ qua docker history)
EXPOSE 3000 22 3306           # khai báo cổng 22 (SSH), 3306 (DB)
CMD ["npm", "start"]           # không USER (chạy root), không nhãn nguồn gốc
```

Những cách kiểm tra sơ sài dễ bị qua mặt. Kiểm tra kiểu "có lệnh `USER`" sẽ bỏ sót Dockerfile nhiều stage, vì `USER` ở stage builder không áp dụng cho image chạy thật. Kiểm tra `EXPOSE` chỉ đánh giá quy ước khai báo image, không phản ánh phơi nhiễm mạng thực tế (phải kiểm tại runtime). Nói chung, metadata tự khai trong image không thay cho bằng chứng thực thi.

Rủi ro nếu để nguyên, không vá. Mỗi vi phạm kéo theo một rủi ro an toàn cụ thể:

- **Chạy bằng root (LEAST-PRIVILEGE):** nếu về sau *giả định* tồn tại thêm một lỗ hổng **thực thi mã (RCE)** trong khi tiến trình chạy bằng root *bên trong* container, thiệt hại sẽ lớn hơn nhiều: kẻ tấn công dễ ghi đè file hệ thống, cài thêm công cụ, và nếu ghép thêm một lỗ hổng nhân hệ điều hành thì có thể **thoát container và leo thang đặc quyền** lên máy chủ. Lưu ý thực nghiệm F01 chỉ chứng minh **vượt xác thực**, chưa phải RCE; chạy non-root (UID 65532) là biện pháp *phòng thủ theo chiều sâu*, khoanh vùng thiệt hại cho những lỗ hổng giả định khác.
- **Khai báo cổng quản trị/CSDL (ATTACK-SURFACE):** `EXPOSE 22/3306` **tự nó không mở** cổng hay khởi động dịch vụ (EXPOSE chỉ mang tính tài liệu), nhưng là **tín hiệu** rằng image dự kiến chạy các dịch vụ này và thường đi kèm publish cổng khi vận hành. Luật chặn khai báo này để giữ *quy ước* image tối giản, hướng người viết Dockerfile không mang theo cổng nhạy cảm; còn phơi nhiễm **thực tế** phải kiểm bằng cấu hình runtime (cổng lắng nghe, publish, network policy), không suy ra từ EXPOSE.
- **Rủi ro chuỗi cung ứng (SUPPLY-CHAIN):** nếu thiếu nhãn nguồn gốc `org.opencontainers.image.source`/`revision`, khi có sự cố sẽ **khó truy vết image bắt nguồn từ commit nào** để điều tra hay thu hồi (đây là *metadata nguồn gốc tự khai*, hỗ trợ truy vết; **chưa đủ để kết luận đạt mức SLSA** — SLSA yêu cầu provenance được *sinh tự động*, mô tả cách build và được phân phối/kiểm chứng). Thêm vào đó, dùng `:latest` hoặc chưa ghim digest khiến image có thể **bị đổi âm thầm** giữa các lần build, làm bản chạy thật khác bản đã kiểm thử; còn lệnh `ADD` thì có thể kéo nội dung từ xa ngoài kiểm soát. Ghim digest cố định tham chiếu image, nhãn nguồn gốc hỗ trợ truy vết, COPY tránh kéo nội dung ngoài kiểm soát; đây là điều kiện cần cho build tái lập, chưa đủ nếu dependency và đầu vào build còn biến đổi.
- **Bí mật nhúng trong image (SECRET-IN-IMAGE):** đặt mật khẩu hay token trong `ENV`/`ARG` khiến bí mật **bị bake vào layer của ảnh**, và bất kỳ ai `docker pull` rồi chạy `docker history` đều đọc được, kể cả khi dòng đó đã bị xóa ở layer sau. Cách đúng là dùng *build secret* hoặc nạp biến môi trường lúc chạy. (Đây là **luật do nhóm bổ sung**, không có trong bản mẫu của lab.)

Sau khi sửa

Việc xử lý gồm **hai phần tách bạch**: (1) **sửa chính file triển khai** cho đúng cấu hình an toàn, và (2) **viết chính sách kiểm tra tự động** để chống tái diễn. Conftest đánh giá cấu hình theo luật Rego, trả PASS/FAIL kèm lý do và mã thoát tương ứng; việc sửa cấu hình được thực hiện riêng trong Dockerfile.

(1) Sửa file Dockerfile triển khai. So với bản lỗi, bản vá (`report/fixtures/Dockerfile.remediated`) khắc phục đúng từng vi phạm tương ứng:

```
FROM node:22-alpine
# THÊM nhãn nguồn gốc OCI  -> đạt SUPPLY-CHAIN (provenance)
LABEL org.opencontainers.image.source="https://github.com/juice-shop/juice-shop" \
      org.opencontainers.image.revision="${VCS_REF}"
RUN addgroup -S -g 10001 juicers && adduser -S -u 10001 -G juicers juicers
COPY --chown=10001:0 . .
USER 10001:10001          # THÊM USER non-root       -> đạt LEAST-PRIVILEGE
EXPOSE 3000               # BỎ cổng 22 và 3306        -> đạt ATTACK-SURFACE
CMD ["npm", "start"]
```

Cụ thể, bản vá thêm lệnh `USER` để chạy non-root, loại hai cổng quản trị/CSDL khỏi `EXPOSE`, bổ sung nhãn nguồn gốc OCI và **không đặt bí mật trong `ENV`/`ARG`** (dùng *build secret* hoặc nạp biến môi trường lúc chạy). Khi phát hành thật, nên ghim base image theo digest, dù hiện luật Rego mới chỉ cảnh báo chứ chưa chặn. Image triển khai thực tế của nhóm (`deploy/Dockerfile.hotfix`) lẫn Dockerfile gốc của Juice Shop đều đã thỏa những yêu cầu này.

(2) Viết chính sách thành mã (Policy as Code) bằng Rego/Conftest (`juice-shop/policy/docker_security.rego`), với mỗi luật được đặt đúng điểm thực thi để tránh các kiểu lách *đã biết* (ví dụ `USER` ở stage builder không tính cho stage cuối, UID root kèm group như `0:1000`). Mã nguồn đầy đủ của bộ luật nằm ở **Phụ lục A**.

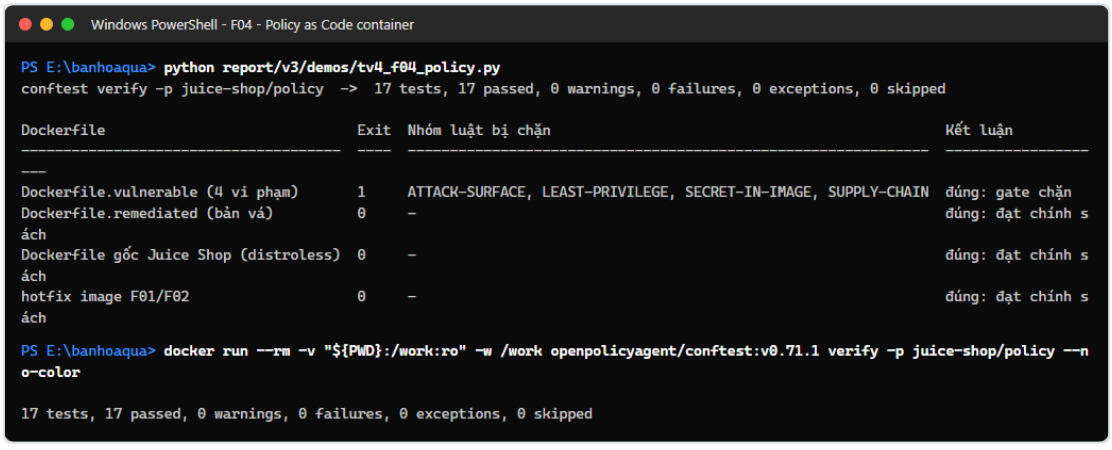
Trong thực nghiệm này, Conftest thực thi **bộ luật Rego do nhóm cung cấp** để đánh giá cấu hình Dockerfile — bản thân công cụ không có luật sẵn, nên chính *bộ luật* mới là cách phát hiện của F04. Bộ kiểm thử Dockerfile gồm **13 ca**, bao gồm các trường hợp root kèm group như `0:1000` và `root:1000`; luật phân tích thành phần UID trước dấu `:` để nhận diện root độc lập với group. Các ca được thiết kế để luật bắt đúng biến thể lách đã biết và không báo nhầm cấu hình hợp lệ; tất cả đạt trong lần chạy `conftest verify` (19/19).

Các luật và tiêu chuẩn đối chiếu:

Bảng 11. Luật kiểm tra cấu hình container — luật yêu cầu gì và dựa trên chuẩn nào

Luật	Luật yêu cầu gì (chặn khi vi phạm)	Tiêu chuẩn (ý nghĩa xem Phụ lục C)
LEAST-PRIVILEGE	Stage cuối **phải** có `USER` không phải root (bắt buộc, `deny`; tách UID trước dấu `:`); khuyến nghị UID số (`warn` nếu không)	CIS Docker 4.1; PCI-DSS 2.2; NĐ 356/2025; Luật ATTTM Đ.19
ATTACK-SURFACE	**Không** được `EXPOSE` cổng quản trị/CSDL (22/23/3306/5432)	CIS Docker 5.8; NIST SP 800-190; Luật ATTTM Đ.19
SUPPLY-CHAIN	**Phải** có nhãn nguồn gốc OCI; **không** dùng `:latest`/`ADD`; nên ghim digest	OCI image-spec; CIS Docker Benchmark §4; NIST SSDF (SLSA: *tham chiếu*, chưa đánh giá đạt)
SECRET-IN-IMAGE *(bổ sung)*	**Không** đặt secret/credential trong `ENV`/`ARG`	CIS Docker 4.10; PCI-DSS 8.2.1; GDPR Đ.32; NĐ 356/2025

Lệnh `conftest verify` chạy toàn bộ chính sách đạt **19/19** ca kiểm thử (13 ca cho chính sách Dockerfile của F04 và 6 ca cho chính sách compose của F03). Kết quả cho thấy `Dockerfile.vulnerable` bị chặn bởi 4 nhóm luật (LEAST-PRIVILEGE, ATTACK-SURFACE, SUPPLY-CHAIN, SECRET-IN-IMAGE), bản vá thì đạt, và Dockerfile gốc distroless của Juice Shop cũng **đạt bộ luật tùy chỉnh này** — lưu ý điều đó *không* đồng nghĩa đạt toàn bộ CIS Docker Benchmark, mà chỉ là đạt các luật đã định nghĩa (Hình 18).



Hình 18. Demo F04: Dockerfile.vulnerable bị chặn bốn nhóm luật, bản vá và Dockerfile gốc đạt chính sách

Nguyên tắc/tiêu chuẩn đối chiếu: GDPR Điều 25(1), 32(1)(d); CIS Docker Benchmark 4.1; Luật 91/2025/QH15 Điều 3 và NĐ 356/2025/NĐ-CP (Điều 12 — điện toán đám mây); Luật ATTTM 2015 Điều 19; TCVN 11930:2017. **Pha DevSecOps:** Build → Release. **Kiểm soát (Policy as Code):** bộ luật Rego chạy như một cổng kiểm tra trong pipeline (đã kiểm chứng cục bộ, chưa nối thành required check).

6. Kết luận

Báo cáo minh họa quy trình **phát hiện – sửa – kiểm chứng** cho hai lỗi mã nguồn (F01 SQL injection ở đăng nhập, F02 Poison Null Byte làm lộ tệp sao lưu), một tình huống phơi nhiễm endpoint giám sát (F03) và một tình huống cấu hình container sai **chủ động dựng** để kiểm tra bộ policy (F04). Toàn bộ được thể hiện dưới dạng Security as Code và Policy as Code trong một chuỗi kiểm soát DevSecOps chạy được, kèm log gốc và mã bằng chứng (mục 4.2).

Chuỗi kiểm soát trải khắp vòng đời — Code/Test (SAST + kiểm thử động), Build/Deploy (Policy as Code), Release (quét image Trivy/Dockle) đến Operate (Compliance as Code + giám sát Prometheus/Grafana). Phần đóng góp của nhóm — trong phạm vi bốn tình huống — gồm: bản vá F01/F02 kèm kiểm thử hồi quy; reverse proxy và giới hạn tốc độ cho F03; bộ luật Rego (Dockerfile + compose, 19 unit test) cho F04; quét image bằng **Trivy (CVE/SCA)** và **Dockle (CIS)** ở pha Release; và **giám sát vận hành bằng Prometheus/Grafana kèm cảnh báo** fire thật ở pha Operate. Kho mã Juice Shop gốc **đã có** CI/CD, CodeQL và tạo SBOM; báo cáo không khẳng định dự án gốc thiếu tự động hóa bảo mật, đóng góp ở đây là các kiểm soát **bổ sung, hướng dữ liệu cá nhân**, hiện chạy cục bộ và **chưa** được nối thành required check bắt buộc trong CI.

Kết quả bị giới hạn bởi tập kiểm thử và bộ policy đã công bố: F04 chỉ chứng minh cấu hình đạt *các luật đã định nghĩa*, không phải đạt toàn bộ CIS Docker Benchmark; MD5 không salt, TLS nội bộ và digest chưa ghim vẫn là rủi ro còn lại; báo cáo đã bổ sung quét **SCA (Trivy)** nhưng **chưa đánh giá DAST** và chưa kết luận production-ready. Đây là những hướng mở rộng tiếp theo, không thuộc phạm vi bốn tình huống của đồ án.

7. Tài liệu tham khảo

- OWASP Juice Shop v20.2.0 — https://owasp.org/www-project-juice-shop/ (truy cập 10/2026).
- GDPR, Regulation (EU) 2016/679, Điều 5, 25, 32.
- Luật Bảo vệ dữ liệu cá nhân 91/2025/QH15 (hiệu lực 01/01/2026), Điều 3 — congbao.chinhphu.vn.
- Nghị định 356/2025/NĐ-CP (hiệu lực 01/01/2026, thay Nghị định 13/2023/NĐ-CP), Điều 9, 12.
- Luật An toàn thông tin mạng 2015 (86/2015/QH13), Điều 16, 17, 19; TCVN 11930:2017.
- PCI-DSS v3.2.1 (đối chiếu lịch sử — đã nghỉ 31/3/2024; bản hiện hành v4.0.1), Yêu cầu 2.2, 3.4, 8.2.1 — pcisecuritystandards.org.

- CIS Docker Benchmark (§4 Container Images, §5 Runtime); OWASP Top 10 2021; NIST SP 800-190; NIST SSDF (SP 800-218); OCI image-spec; SLSA v1.x (slsa.dev).
- Công cụ (phiên bản dùng trong thực nghiệm): Semgrep 1.180.0, Gitleaks v8.30.1, Conftest/OPA v0.71.1, node:test + Supertest, Trivy 0.76.1, Dockle 0.4.15, Prometheus 3.15.1, Grafana 13.3.0.
- Phương pháp luận tham khảo: *Security as Code with OWASP Juice Shop & Docker*; *Policy as Code with Conftest*.

Phụ lục A — Mã nguồn chính sách Dockerfile (docker_security.rego)

Nguồn: kế thừa cấu trúc và các luật nền tảng từ tài liệu hướng dẫn *Policy as Code with Conftest* (Appendix A.1); phần nhóm điều chỉnh/bổ sung: bỏ luật HEALTHCHECK, gộp provenance vào SUPPLY-CHAIN, và **tự viết luật** `SECRET-IN-IMAGE` (xem mục 5.4). Namespace `dockerfile`, 8 luật `deny`; bộ kiểm thử `docker_security_test.rego` có 13 ca. Mỗi thông báo vi phạm trích kèm điều khoản chuẩn quốc tế và Việt Nam (Luật 91/2025/QH15 Đ.3; NĐ 356/2025/NĐ-CP; Luật ATTTM 2015 Đ.19). Chạy: `conftest test <Dockerfile> -p juice-shop/policy --namespace dockerfile`.

```
# Policy as Code cho Dockerfile của Juice Shop — theo mô hình "Policy as Code with Conftest".
# Mỗi luật đặt tại điểm thực thi đúng thay vì các "original check" dễ bị qua mặt:
#   Least privilege  -> USER ở STAGE CUỐI, UID số, không phải root (thay vì "có USER bất kỳ trong
file")
#   Firewall/RBAC    -> cổng quản trị/DB không được EXPOSE (giảm bề mặt tấn công)
#   Supply chain     -> metadata nguồn gốc (nhãn OCI source+revision; hỗ trợ truy vết, CHƯA đánh
giá đạt SLSA) + không :latest, không ADD, cảnh báo base chưa ghim digest
#   Secret hygiene   -> KHÔNG nhúng bí mật (ENV/ARG) vào image (cải tiến của nhóm, ngoài bản mẫu
lab)
# Mỗi thông báo vi phạm trích sẵn điều khoản chuẩn áp dụng — quốc tế (CIS Docker, PCI-DSS, NIST,
OCI/SLSA)
# và Việt Nam (Luật Bảo vệ dữ liệu cá nhân 91/2025/QH15 Điều 3 — nguyên tắc bảo mật, an toàn; Nghị
định
# 356/2025/NĐ-CP quy định chi tiết, hiệu lực 01/01/2026 thay Nghị định 13/2023; Luật An toàn thông
tin mạng
# 2015 Điều 19 — biện pháp kỹ thuật bảo đảm an toàn thông tin cá nhân, tuân thủ tiêu chuẩn kỹ
thuật).
#   chạy: conftest test <Dockerfile> -p policy/ --namespace dockerfile
package dockerfile

import rego.v1

# Danh sách cổng quản trị/CSDL — giữ theo Appendix A.1 của lab (SSH, telnet, Docker daemon, MySQL,
Postgres, Redis, Mongo).
forbidden_ports := {"22", "23", "2375", "2376", "3306", "5432", "6379", "27017"}

required_labels := {"org.opencontainers.image.source", "org.opencontainers.image.revision"}

# UID là phần trước dấu ':' trong 'USER uid[:gid]'. Nhận diện root ('0' hoặc 'root') độc lập với
group
# kèm theo, nên 'USER 0:1000' cũng bị nhận diện là root.
uid_of(u) := split(u, ":")[0]

# Chỉ STAGE CUỐI tạo ra image chạy thật; USER ở stage builder không áp dụng cho runtime.
final_stage := max([ins.Stage | some ins in input])

final_users := [ins.Value[0] | some ins in input; ins.Stage == final_stage; ins.Cmd == "user"]

last_user := final_users[count(final_users) - 1] if count(final_users) > 0

# LEAST-PRIVILEGE ------------------------------------------------------------
deny contains msg if {
        count(final_users) == 0
        msg := "LEAST-PRIVILEGE: final stage has no explicit USER; policy requires an explicit non-
root USER (otherwise the container may run as root). (CIS Docker 4.1; PCI-DSS 2.2; NIST SP 800-190;
Luật 91/2025/QH15 Đ.3; Luật ATTTM 2015 Đ.19)"
```

```
}

deny contains msg if {
        uid_of(last_user) in {"root", "0"}
        msg := sprintf("LEAST-PRIVILEGE: final USER '%s' runs as root (UID 0), regardless of group.
(CIS Docker 4.1; PCI-DSS 2.2; NĐ 356/2025/NĐ-CP; Luật 91/2025/QH15 Đ.3)", [last_user])
}

warn contains msg if {
        last_user
        not uid_of(last_user) in {"root", "0"}
        not regex.match(`^[0-9]+(:[0-9]+)?$`, last_user)
        msg := sprintf("LEAST-PRIVILEGE: final USER '%s' is not a fixed numeric UID. (CIS Docker
4.1)", [last_user])
}

# ATTACK-SURFACE (firewall / RBAC) ----------------------------------------
port_number(raw) := split(raw, "/")[0] # bỏ hậu tố /tcp, /udp

deny contains msg if {
        some ins in input
        ins.Cmd == "expose"
        some raw in ins.Value
        port_number(raw) in forbidden_ports
        msg := sprintf("ATTACK-SURFACE: EXPOSE '%s' covers administrative/database port %s. (CIS
Docker 5.8; NIST SP 800-190; Luật ATTTM 2015 Đ.19)", [raw, port_number(raw)])
}

# SUPPLY-CHAIN — provenance khai báo (traceability; không thay cho chữ ký/attestation) --
label_blob := concat(" ", [concat(" ", ins.Value) | some ins in input; ins.Cmd == "label"])

deny contains msg if {
        some req in required_labels
        not contains(label_blob, req)
        msg := sprintf("SUPPLY-CHAIN: missing provenance label '%s' (metadata nguồn gốc hỗ trợ truy
vết; chưa đánh giá đạt mức SLSA). (OCI image-spec; NIST SSDF)", [req])
}

# SUPPLY-CHAIN — base image (không :latest/ADD, ghim digest) -----------------
deny contains msg if {
        some ins in input
        ins.Cmd == "from"
        endswith(ins.Value[0], ":latest")
        msg := sprintf("SUPPLY-CHAIN: base image '%s' uses :latest; pin a version or digest. (CIS
Docker Benchmark §4 Container Images; NIST SSDF)", [ins.Value[0]])
}

deny contains msg if {
        some ins in input
        ins.Cmd == "add"
        msg := "SUPPLY-CHAIN: use COPY instead of ADD to avoid remote/implicit extraction. (CIS
Docker 4.9)"
}

warn contains msg if {
        some ins in input
        ins.Cmd == "from"
        not contains(ins.Value[0], "@sha256:")
        not endswith(ins.Value[0], ":latest")
        msg := sprintf("SUPPLY-CHAIN: base image '%s' is not pinned by digest (digest pin khác với
Content Trust/CIS 4.5). (CIS Docker Benchmark §4; NIST SSDF)", [ins.Value[0]])
}

# SECRET-IN-IMAGE — cải tiến của nhóm (ngoài bản mẫu lab) --------------------
# Bí mật đặt trong ENV/ARG bị bake vào layer ảnh và lộ qua `docker history`. Ánh xạ
# luật secret của compose (F03) xuống tầng Dockerfile. ENV parse thành [key, value, "=", ...];
```

```
# ARG parse thành ["KEY=VALUE"].
secret_key := `(?i)(password|passwd|secret|token|api[_-]?key|access[_-]?key|private[_-]?
key|credential)`

deny contains msg if {
        some ins in input
        ins.Cmd == "env"
        some i, key in ins.Value
        i % 3 == 0
        i + 1 < count(ins.Value)
        regex.match(secret_key, key)
        ins.Value[i + 1] != ""
        msg := sprintf("SECRET-IN-IMAGE: ENV '%s' hardcodes a credential into the image (exposed
via docker history); use build secrets/runtime env. (CIS Docker 4.10; GDPR Art.32; PCI-DSS 8.2.1;
NĐ 356/2025/NĐ-CP; Luật 91/2025/QH15 Đ.3)", [key])
}

deny contains msg if {
        some ins in input
        ins.Cmd == "arg"
        some tok in ins.Value
        contains(tok, "=")
        regex.match(secret_key, split(tok, "=")[0])
        split(tok, "=")[1] != ""
        msg := sprintf("SECRET-IN-IMAGE: ARG '%s' bakes a credential default into the image. (CIS
Docker 4.10; GDPR Art.32; PCI-DSS 8.2.1; NĐ 356/2025/NĐ-CP; Luật 91/2025/QH15 Đ.3)", [split(tok,
"=")[0]])
}
```

Phụ lục B — Mã nguồn chính sách docker-compose (compose.rego)

Nguồn: nhóm tự viết cho docker-compose — mẫu tham khảo chỉ áp dụng trên manifest Kubernetes, nhóm chuyển các ý tưởng least-privilege/firewall/secret sang định dạng triển khai thật. Namespace `compose`, 5 luật `deny` (chặn backend mở cổng host, bind ngoài loopback, privileged, mount docker.sock, secret nội tuyến); bộ kiểm thử `compose_test.rego` có 6 ca. Thông báo vi phạm trích kèm điều khoản chuẩn quốc tế và Việt Nam (Luật 91/2025/QH15 Đ.3; NĐ 356/2025/NĐ-CP; Luật ATTTM 2015 Đ.19). Chạy: `conftest test docker-compose.yml -p juice-shop/policy --namespace compose`.

```
package compose

import rego.v1

# Input: file docker-compose (conftest tự đọc YAML).
# ports chấp nhận cả 3 dạng của Compose: "127.0.0.1:3102:8080", 3000, {target, published, host_ip}.
# Thông báo vi phạm trích điều khoản chuẩn quốc tế (GDPR, PCI-DSS, CIS Docker) và Việt Nam
# (Luật Bảo vệ dữ liệu cá nhân 91/2025/QH15 Điều 3 và Nghị định 356/2025/NĐ-CP; Luật An toàn thông
tin mạng 2015 Điều 19).

host_ip(p) := split(p, ":")[0] if {
        is_string(p)
        count(split(p, ":")) == 3
}

host_ip(p) := "" if {
        is_string(p)
        count(split(p, ":")) < 3
}

host_ip(p) := "" if is_number(p)

host_ip(p) := object.get(p, "host_ip", "") if is_object(p)

ports(svc) := object.get(svc, "ports", [])

# Dịch vụ được thiết kế để nhận kết nối từ host (reverse proxy, dashboard) phải tự khai báo nhãn.
edge(svc) if object.get(object.get(svc, "labels", {}), "lab.exposure", "") == "edge"

# F03 / NET-02: backend không được publish cổng. Nếu backend có cổng host, mọi route (kể cả
/metrics)
# có thể bị truy cập vòng qua proxy, làm vô hiệu luật chặn /metrics và giới hạn tốc độ ở proxy.
deny contains msg if {
        some name, svc in input.services
        count(ports(svc)) > 0
        not edge(svc)
        msg := sprintf("[F03 NET-02] service '%s' publishes ports but is not an edge service (label
lab.exposure=edge); backend must only be reachable through the proxy (Luật ATTTM 2015 Đ.19; NĐ
356/2025/NĐ-CP; Luật 91/2025 Đ.3)", [name])
}

# NET-01 (GDPR Art.32): cổng publish chỉ được bind loopback trong lab.
deny contains msg if {
        some name, svc in input.services
        some p in ports(svc)
        host_ip(p) != "127.0.0.1"
```

```
        msg := sprintf("[GDPR Art.32 NET-01] service '%s' publishes %v on all interfaces (NĐ
356/2025/NĐ-CP; Luật ATTTM 2015 Đ.19)", [name, p])
}

deny contains msg if {
        some name, svc in input.services
        svc.privileged == true
        msg := sprintf("[CIS Docker 5.4] service '%s' runs privileged", [name])
}

deny contains msg if {
        some name, svc in input.services
        some v in object.get(svc, "volumes", [])
        contains(sprintf("%v", [v]), "docker.sock")
        msg := sprintf("[CIS Docker 5.31] service '%s' mounts the Docker socket", [name])
}

# Biến môi trường có thể là danh sách "KEY=value" hoặc map {KEY: value}; *_FILE trỏ tới secret thì
được phép.
env(svc) := [e | some e in svc.environment] if is_array(svc.environment)

env(svc) := [sprintf("%s=%v", [k, v]) | some k, v in svc.environment] if is_object(svc.environment)

env(svc) := [] if not svc.environment

deny contains msg if {
        some name, svc in input.services
        some e in env(svc)
        regex.match(`(?i)^[A-Z0-9_]*(PASSWORD|SECRET|TOKEN|API_KEY)=.+`, e)
        msg := sprintf("[PCI-DSS 8.2.1 / GDPR Art.32 / NĐ 356/2025/NĐ-CP] service '%s' has an
inline secret in environment: %s", [name, split(e, "=")[0]])
}

drops_all_caps(svc) if "ALL" in object.get(svc, "cap_drop", [])

warn contains msg if {
        some name, svc in input.services
        not drops_all_caps(svc)
        msg := sprintf("[CIS Docker 5.3] service '%s' does not drop capabilities", [name])
}

warn contains msg if {
        some name, svc in input.services
        not svc.read_only
        msg := sprintf("[CIS Docker 5.12] service '%s' has a writable root filesystem", [name])
}
```

Phụ lục C — Giải thích tiêu chuẩn và điều khoản tham chiếu

Bảng tra ý nghĩa (**yêu cầu gì**) của mọi tiêu chuẩn/điều khoản được trích trong báo cáo. Phần thân chỉ ghi mã; ý nghĩa tra tại đây.

Bảng 12. Tra cứu — mỗi tiêu chuẩn/điều khoản tham chiếu yêu cầu gì

Điều khoản	Yêu cầu / ý nghĩa tóm gọn
GDPR Điều 5(1)(f)	Nguyên tắc *toàn vẹn & bảo mật*: xử lý dữ liệu một cách an toàn
GDPR Điều 25	Bảo vệ dữ liệu *từ thiết kế* (khoản 1) và *theo mặc định* (khoản 2)
GDPR Điều 32	Biện pháp kỹ thuật & tổ chức bảo đảm an toàn khi xử lý — (1)(b) bí mật/toàn vẹn, (1)(d) kiểm thử & đánh giá định kỳ
OWASP A02:2021 / CWE-327	*Cryptographic Failures* — lỗi mật mã (vd băm mật khẩu bằng thuật toán yếu)
OWASP A03:2021 / CWE-89	*Injection* — tiêm nhiễm dữ liệu vào câu lệnh (SQL injection)
OWASP A01:2021 / CWE-158	*Broken Access Control* — xử lý sai null byte (`%00`) làm vượt kiểm tra loại tệp (F02)
CWE-180	Kiểm tra đầu vào *trước khi* chuẩn hóa — tên được kiểm tra khác tên thực sự dùng
PCI-DSS 2.2	Cấu hình tham số an toàn hệ thống, chạy với đặc quyền tối thiểu
PCI-DSS 3.4	Làm số thẻ (PAN) không đọc được ở mọi nơi lưu trữ
PCI-DSS 8.2.1	Không lưu thông tin xác thực (credential) ở dạng đọc được
CIS Docker 4.1	Chạy container bằng user riêng, **không phải root**
CIS Docker 4.5	**Content Trust** — bật chữ ký số cho ảnh (`DOCKER_CONTENT_TRUST`). *Ghim digest/tránh* `:latest` là kiểm soát riêng thuộc nhóm quản lý ảnh nền (§4), không phải 4.5
CIS Docker 4.9	Dùng `COPY` thay cho `ADD` (tránh kéo nội dung từ xa ngoài kiểm soát)
CIS Docker 4.10	Không lưu bí mật trong Dockerfile
CIS Docker 5.3	Loại bỏ Linux capabilities không cần (`cap_drop: ALL`)
CIS Docker 5.4	Không chạy container ở chế độ `privileged`
CIS Docker 5.8	Chỉ mở đúng cổng cần thiết (không mở cổng quản trị/CSDL)
CIS Docker 5.12	Gắn hệ thống tệp gốc ở chế độ chỉ đọc (`read_only`)
CIS Docker 5.31	Không mount Docker socket vào container
OCI image-spec	Chuẩn định dạng ảnh container + nhãn metadata nguồn gốc
SLSA	Khung toàn vẹn chuỗi cung ứng. Mức Build L1 yêu cầu provenance được *sinh tự động* và phân phối; nhãn OCI tự khai trong báo cáo **chưa đạt** L1, chỉ dùng để tham chiếu
NIST SP 800-190	Hướng dẫn an toàn cho container
Luật 91/2025/QH15 Điều 3	Nguyên tắc bảo vệ dữ liệu cá nhân (hợp pháp, đúng mục đích, chính xác, bảo mật – an toàn). Hiệu lực 01/01/2026

Điều khoản	Yêu cầu / ý nghĩa tóm gọn
NĐ 356/2025/NĐ-CP	Quy định chi tiết biện pháp bảo vệ dữ liệu cá nhân (quản lý & kỹ thuật: mã hóa, xác thực, kiểm soát truy cập, giám sát); Điều 12 — điện toán đám mây. Thay Nghị định 13/2023 từ 01/01/2026
Luật ATTTM 2015 Điều 16	Nguyên tắc bảo vệ thông tin cá nhân
Luật ATTTM 2015 Điều 17	Không chia sẻ thông tin cá nhân cho bên thứ ba khi chưa được đồng ý
Luật ATTTM 2015 Điều 19	Biện pháp quản lý & kỹ thuật bảo đảm an toàn thông tin cá nhân; tuân thủ tiêu chuẩn kỹ thuật
TCVN 11930:2017	Yêu cầu cơ bản về an toàn hệ thống thông tin theo cấp độ